



Dipartimento di Ingegneria e Scienze dell'informazione

Progetto:

UrbanLock

Gruppo:

G26

Titolo del documento:

Sviluppo

Doc. Name	D2-UrbanLockSviluppoProgetto	Doc. Number	D2 V 2.2
Description	Documento di sviluppo dell'applicazione		

INDICE

1. Scopo del documento	1
2. Web APIS	2
e3. Implementation	2
4. FrontEnd	6
5. Deployment	17
6. CI/CD (Continuous Integration / Continuous Deployment)	19

1. Scopo del documento

Il presente documento riporta tutte le informazioni necessarie per lo sviluppo dell'applicazione UrbanLock. In particolare, presenta tutti gli artefatti necessari per realizzare i servizi di gestione dei locker, depositi, prestiti, donazioni e segnalazioni con l'applicazione UrbanLock.

Partendo dalla presentazione delle web APIs del servizio web, proseguendo con l'organizzazione del codice, il modello dati, e ulteriori informazioni su testing. Infine, una sezione dedicata al front-end e una agli aspetti di deployment.

2. Web APIS

Le API sono state documentate secondo le specifiche openAPI3 e la documentazione è consultabile pubblicamente su [swagger/apiary/altro](https://urbanlockapi.docs.apiary.io/#) al seguente indirizzo:

<https://urbanlockapi.docs.apiary.io/#>

[La specifica delle API è disponibile nel repository al link](#)

3. Implementation

L'applicazione è stata sviluppata utilizzando le seguenti tecnologie: Node.js con Express.js per il backend e Flutter per il frontend (app mobile e console admin), per la gestione dei dati abbiamo utilizzato MongoDB. Lo stack tecnologico è stato motivato dal materiale fornito nel corso / conoscenze pregresse / esigenze del progetto (supporto multi-piattaforma mobile, architettura modulare, integrazione IoT) / altro

Repository Organization

Il codice del progetto (<https://github.com/UrbanLock/root>) è organizzato secondo la seguente struttura:

- `/backend`	cartella backend Node.js
- `/src`	file sorgenti
- `/config`	configurazioni (database, env, tls)
- `/controllers`	logica business per ogni risorsa
- `/models`	modelli dati mongoose
- `/routes`	endpoint per le risorse
- `/admin`	routes amministrative
- `/middleware`	middleware di autenticazione e gestione errori

- `/services`	servizi business (auth, notifications)
- `/utils`	utility (logger, photo storage)
- `server.js`	applicazione Express.js
- `/uploads`	file caricati (foto)
- `/certificates`	certificati TLS/SSL
- `oas3.yaml`	specifica OpenAPI 3.0 per documentazione API
- `package.json`	file di configurazione del progetto npm
- `package-lock.json`	lock file dipendenze npm
- `.env`	file di configurazione variabili d'ambiente
- `.env.example`	esempio di file di configurazione variabili d'ambiente
- `/frontend`	codice per la parte del front-end
- `/app`	app mobile Flutter (utenti)
- `/console`	console admin Flutter
- `/docs`	documentazione progetto
- `/Deliverables`	deliverable D1-D4
- `README.md`	documentazione principale
- `.gitignore`	configurazione repository git

Branching strategy e organizzazione del lavoro

Lo sviluppo ha visto una collaborazione attiva tra i membri del gruppo. Ci siamo divisi il lavoro in questo modo:

- **Backend API:** Sviluppo delle API REST per gestione locker, depositi, prestiti, donazioni, segnalazioni
- **Frontend Mobile App:** Sviluppo app Flutter per utenti finali con mappa interattiva, gestione depositi, notifiche
- **Frontend Console Admin:** Sviluppo console amministrativa Flutter per operatori e admin con dashboard analytics
- **Database & Models:** Definizione schema MongoDB e modelli Mongoose per tutte le entità
- **Autenticazione & Sicurezza:** Implementazione JWT, middleware auth, validazione input con express-validator

In totale abbiamo eseguito più di 127 commit distribuiti tra i membri in questo modo:

- Gurhart Singh Gill:	39 commit (backend API, autenticazione)
- Tommaso Cattelan:	85 commit (frontend mobile app)
- Harmeet Singh:	36 commit (frontend console admin, database)

Lo sbilanciamento nel numero di commit tra i membri del gruppo non riflette una differenza nel carico di lavoro o nel contributo effettivo al progetto, ma è principalmente legato a fattori tecnici e metodologici.

In particolare:

- **Differente granularità dei commit:**
Alcuni membri hanno adottato una strategia di commit molto frequenti e di piccola

entità (es. fix puntuali, piccoli miglioramenti UI, refactoring incrementali), mentre altri hanno preferito effettuare commit meno frequenti ma più consistenti, comprendenti intere funzionalità o blocchi di lavoro completi.

- **Natura delle attività svolte:**

Lo sviluppo del frontend mobile in Flutter ha richiesto numerosi interventi incrementali su UI, layout, gestione stato e debugging grafico, che hanno portato naturalmente a un numero elevato di commit.

Al contrario, lo sviluppo del backend API e dell'autenticazione ha previsto implementazioni più strutturate e concentrate, spesso sviluppate localmente e poi integrate in un numero ridotto di commit più corposi.

- **Gestione di librerie e dipendenze:**

In alcune fasi iniziali del progetto sono stati erroneamente committati file di librerie esterne o file generati automaticamente, aumentando artificialmente il numero totale di commit in determinate aree.

- **Collaborazione e revisione:**

Tutti i membri hanno partecipato attivamente alla progettazione, revisione del codice e discussione delle scelte architetturali, anche quando non risultano direttamente come autori dei commit.

Dependencies

Il progetto npm si basa sui seguenti moduli esterni:

- **cors:** Per il supporto alle chiamate cors
- **dotenv:** Gestione variabili d'ambiente in ambiente dev da file .env
- **express:** Framework web per il backend
- **express-validator:** Validazione input richieste
- **helmet:** Sicurezza HTTP headers
- **jsonwebtoken:** Autenticazione JWT
- **mongoose:** Libreria per interfacciarsi con mongoDB
- **winston:** Sistema di logging

E le seguenti dipendenze di sviluppo:

- **dotenv:** Gestione variabili d'ambiente in ambiente dev da file .env
- **Jest:** Testing
- **Supertest:** Testing endpoint express

Database

Per la gestione dei dati utili all'applicazione abbiamo definito le seguenti strutture dati tramite l'utilizzo di mongoose.

- **User** (Collezione: utente)
- **Locker** (Collezione: locker)
- **Cell** (Collezione: cella)
- **Noleggior** (Collezione: noleggior)
- **Donazione** (Collezione: donazione)
- **Segnalazione** (Collezione: segnalazione)
- **Notifica** (Collezione: notifica)

Testing

Le api e il relativo codice presentano una test-suite che consente di verificarne il corretto funzionamento. I test sono utilizzati all'interno della configurazione di CI/CD e vengono eseguiti automaticamente su ogni push e pull request tramite GitHub Actions.

L'implementazione dei test è organizzata in file `test.js` che affiancano i vari file dell'implementazione nella struttura `/backend/src/controllers/__tests__`, `/backend/src/routes/__tests__`, `/backend/src/services/__tests__`.

N. Test Case	Descrizione	Test Data	Precondizioni	Dipendenze	Risultato Atteso	Risultato	Note
1	Health Check	GET /health	---	---	200, success=true,	PASS	health.test.js
2	Creazione account	CF=RSSMR A80A01H5 01U, tipo=spid, nome=Mario, cognome=Rossi	CF mai usato	---	201, success=true, user+token	PASS	auth.test.js
3	Account senza CF	CF vuoto, tipo=spid	---	---	400, success=false, nessun account	PASS	auth.test.js
4	Tipo	CF=RSSMR	---	---	400,	PASS	auth.test.js

	autenticazione non valido	A80A01H501U, tipo=invalid			success=false, nessun account		
5	Lista locker senza filtri	GET /api/v1/lockers	---	---	200, success=true, tutti i locker	PASS	lockers.test.js
6	Lista locker per tipo	GET /api/v1/lockers?type=sportivi	locker sportivi presenti	---	200, success=true, solo sportivi	PASS	lockers.test.js
7	Dettaglio locker inesistente	GET /api/v1/lockers/INVALID-ID	---	---	404, success=false	PASS	lockers.test.js
8	Deposito non autenticato	POST /api/v1/deposits {lockerId:LOCK-001, cellId:CELL-001} senza Authorization	---	---	401, success=false, nessun deposito	PASS	deposits.test.js
9	Refresh token vuoto	POST /api/v1/auth/refresh {refreshToken:""}	---	---	400, success=false, nessun rinnovo	PASS	auth.test.js
10	Info utente senza auth	GET /api/v1/auth/me senza Authorization	---	---	401, success=false, nessuna info utente	PASS	auth.test.js

4. FrontEnd

Il FrontEnd fornisce le funzionalità per visualizzare, inserire e gestire i dati per l'applicazione UrbanLock. In particolare, l'applicazione è composta da una Home Page, una pagina per la gestione dei locker, una pagina per la gestione dei depositi, una pagina per le donazioni, una pagina per le segnalazioni, e una pagina per le notifiche.

FrontEnd lato Utente

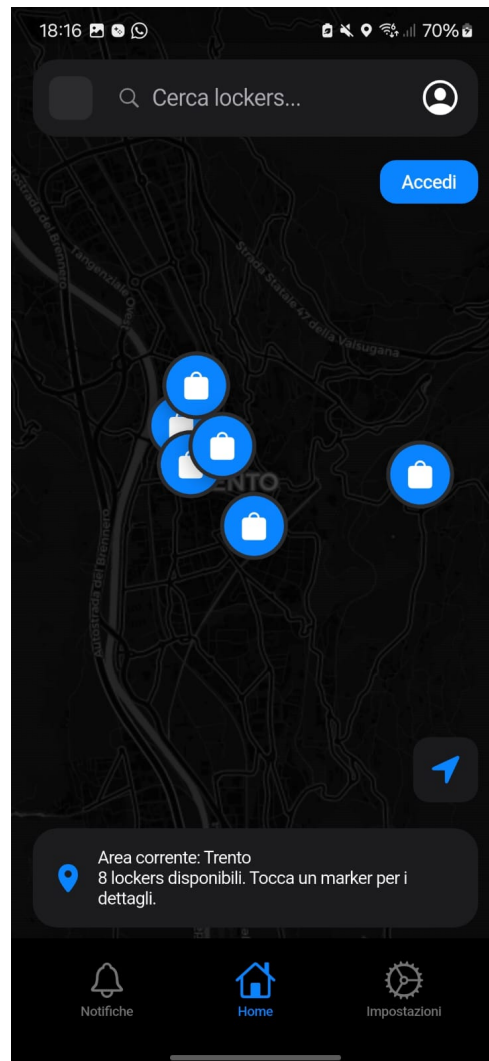


Figura 1 Home Page:

La Figura 1 mostra la home page dell'applicazione. Da qui, l'utente può autenticarsi e navigare verso le altre pagine dell'app. La home page mostra una mappa interattiva con i locker disponibili nella città di Trento, permettendo all'utente di cercare e filtrare i locker per categoria, tipo e distanza.

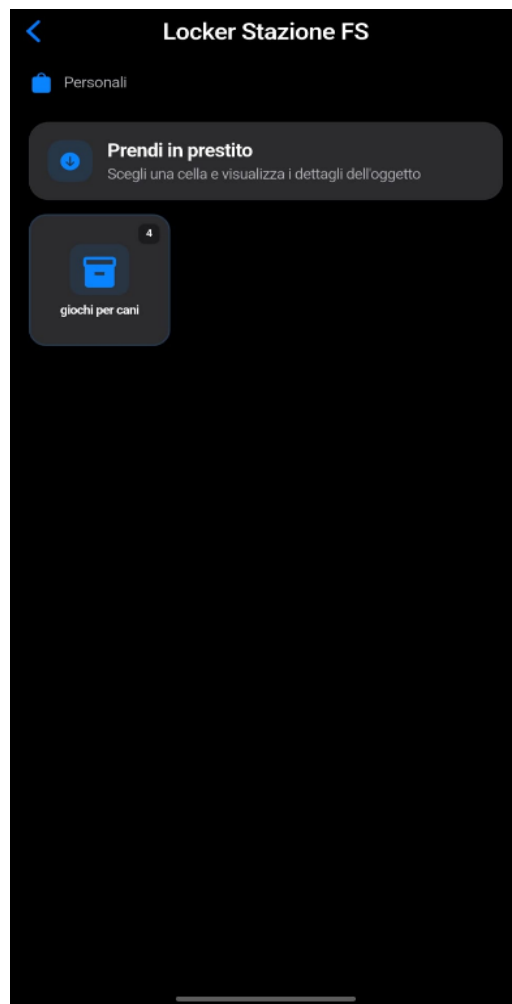


Figura 2 Schermata locker (Locker Screen):

La Figura 2 mostra la lista dei locker disponibili che possono essere visualizzati dall'utente. È anche possibile visualizzare i dettagli di un locker specifico, incluse le celle disponibili e le statistiche di utilizzo.

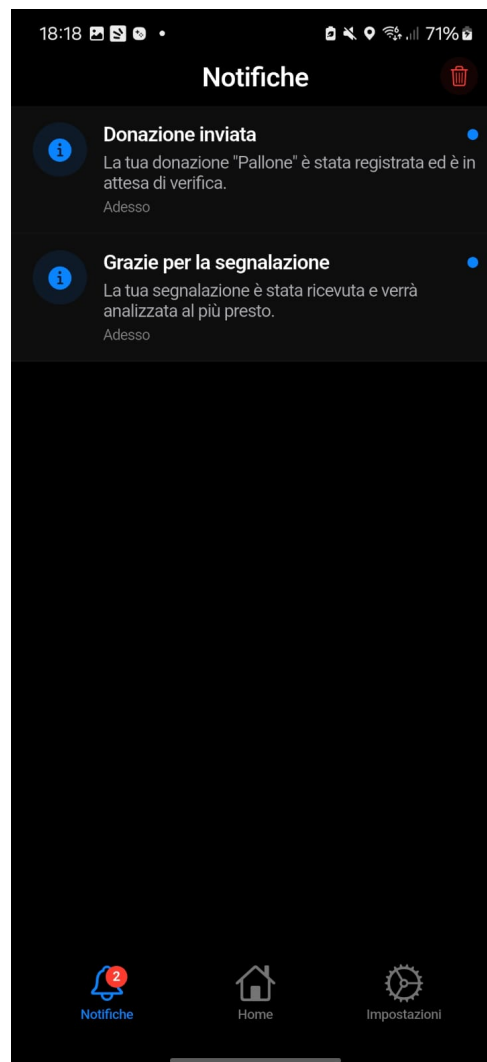


Figura 3 Schermata notifiche (Notifications Screen):

La Figura 3 mostra la schermata delle notifiche dell'applicazione. In questo esempio l'utente visualizza notifiche relative a una donazione inviata e a una segnalazione ricevuta, con indicazione dello stato e del momento di invio. Dal menu in basso può passare rapidamente alle sezioni Home e Impostazioni.

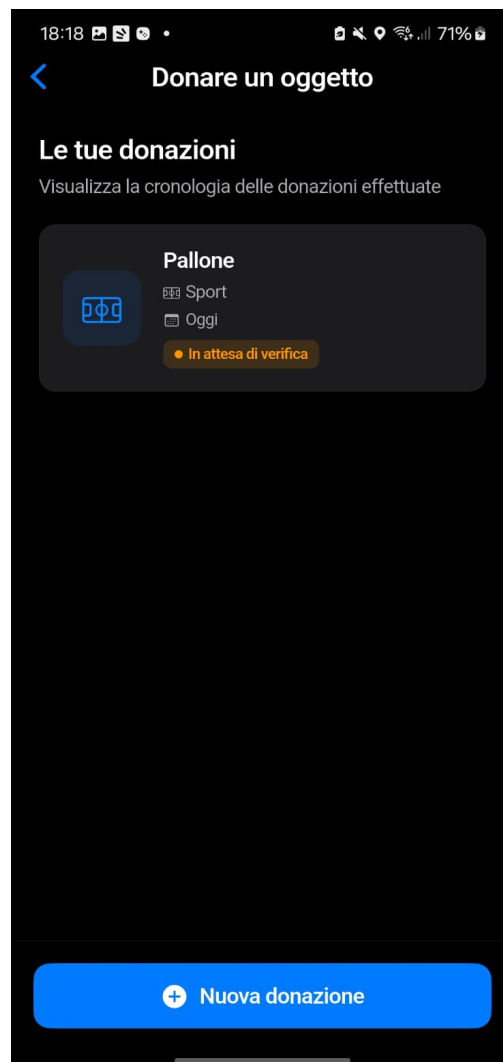


Figura 4 Storico donazioni (Donations History):

La Figura 4 mostra la schermata per la gestione delle donazioni, con l'elenco degli oggetti donati e il relativo stato (ad esempio "In attesa di verifica"). L'utente può consultare la cronologia delle donazioni effettuate e avviare una nuova donazione tramite l'apposito pulsante in basso.



Figura 5 Schermata di login (Login Screen):

La Figura 5 mostra la schermata di accesso all'applicazione UrbanLock. L'utente viene accolto da un messaggio di benvenuto e può autenticarsi tramite SPID o CIE, selezionando il metodo preferito per accedere a tutti i servizi offerti dall'app.

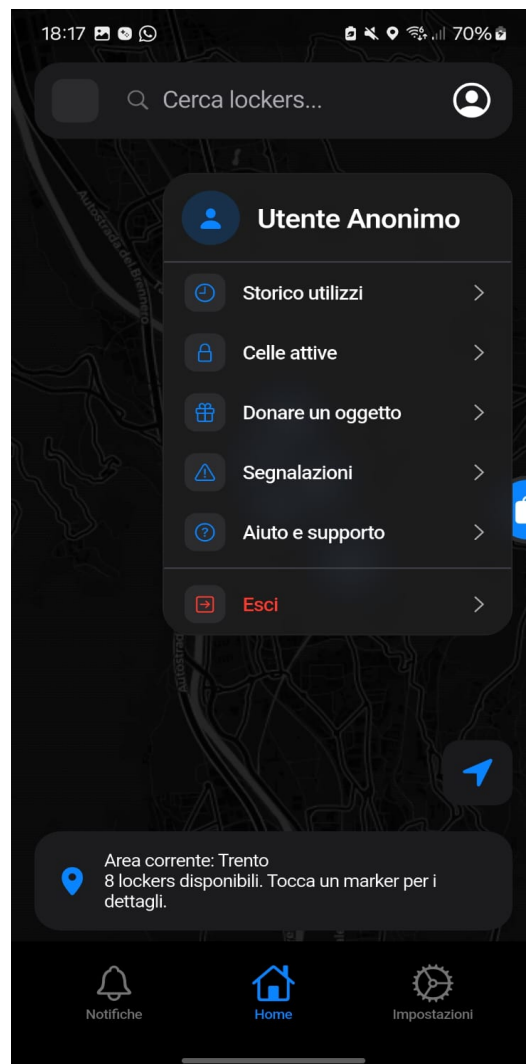


Figura 6 Menu utente sulla Home (User Menu on Home) :

La Figura 6 mostra il menu utente aperto sulla mappa principale. Anche come utente anonimo è possibile accedere rapidamente alle sezioni Storico utilizzi, Celle attive, Donare un oggetto, Segnalazioni, Aiuto e supporto, oltre all'azione di logout, mantenendo sempre visibile la mappa dei locker sullo sfondo.

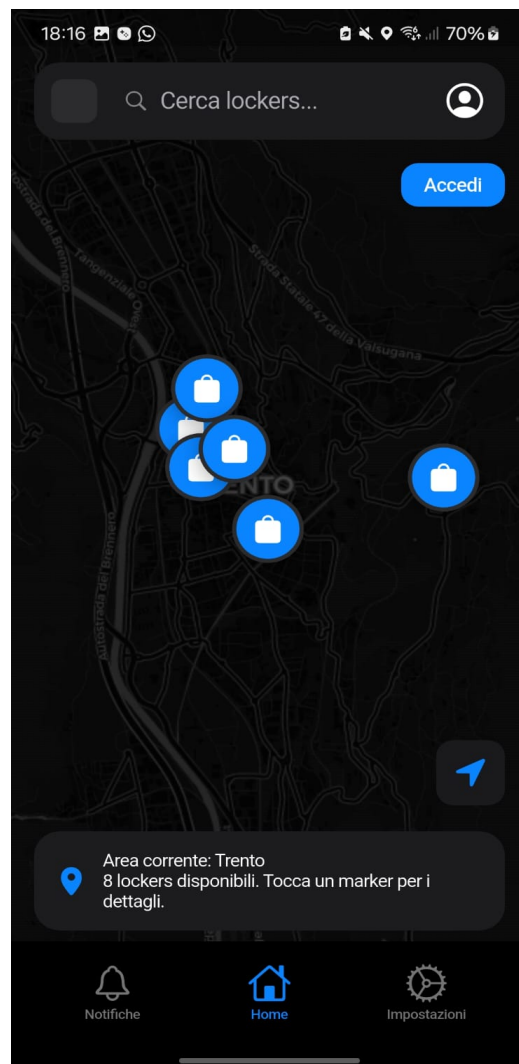


Figura 7 Mappa dei locker (Lockers Map Screen) :

La Figura 7 mostra la schermata principale con la mappa dei locker disponibili nell'area corrente (ad esempio Trento). L'utente può cercare i locker tramite barra di ricerca, visualizzare i marker sulla mappa e leggere un riepilogo sul numero di locker disponibili; tramite il pulsante "Accedi" può autenticarsi per sbloccare funzionalità aggiuntive.

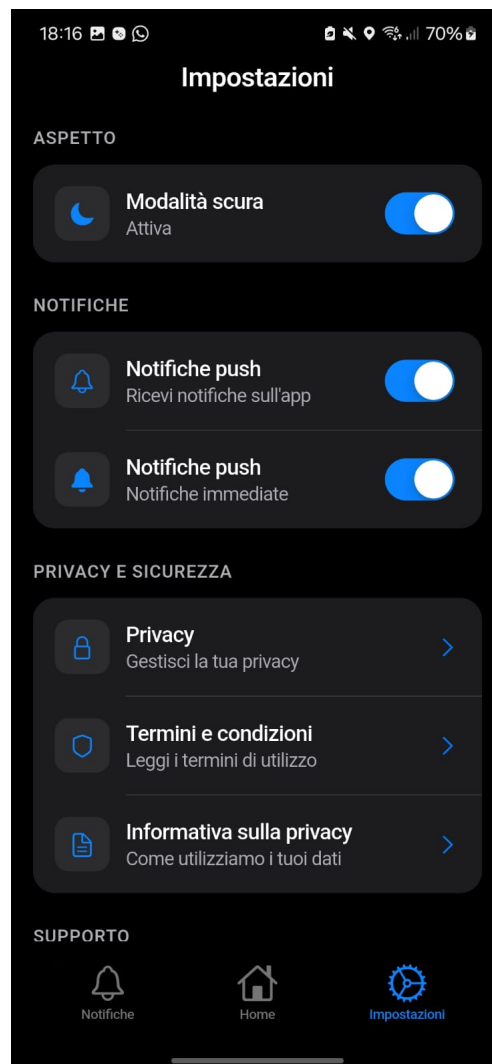


Figura 8 Schermata impostazioni (Settings Screen):

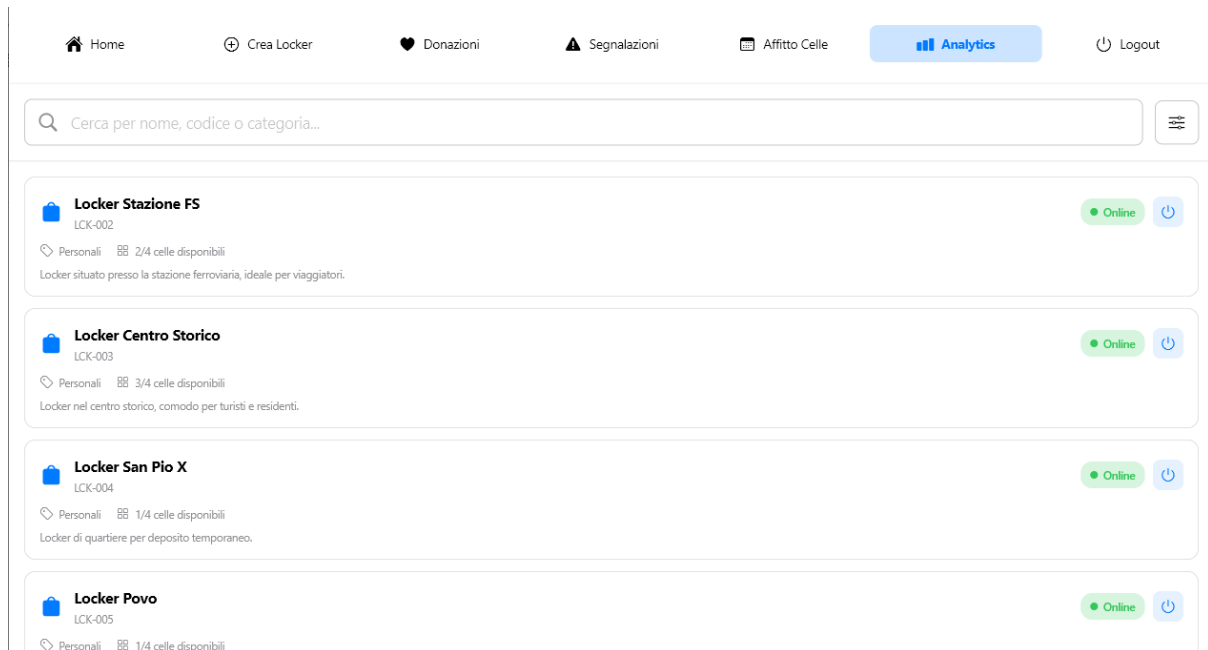
La Figura 8 mostra la schermata delle impostazioni dell'app. Da qui l'utente può abilitare o disabilitare la modalità scura, gestire le notifiche push, consultare le sezioni di privacy e sicurezza (privacy, termini e condizioni, informativa sulla privacy) e accedere alle opzioni di supporto.



Figura 9 Onboarding iniziale (Onboarding - Find Closest Locker) :

La Figura 9 mostra una delle schermate del flusso di onboarding, che introduce l'utente alle funzionalità principali dell'app. In questo esempio viene spiegato come trovare il locker più vicino tramite la mappa interattiva, con un pulsante "Avanti" che consente di proseguire con i passaggi successivi della guida.

Frontend lato operatore



Homepage - Gestione Locker

La homepage visualizza un elenco filtrabile di locker con stato real-time (disponibile, occupato, manutenzione), barra di ricerca e navigazione laterale (Home, Categorie, Donazioni, Segnalazioni, Cella, Analytics, Logout). Cliccando su un locker specifico, si espandono automaticamente le celle contenute al suo interno, mostrando disponibilità individuale. Cambiare lo stato di un locker aggiorna tutte le sue celle in cascata; modificare una singola cella impatta solo quella. Per disattivazioni (sia locker che celle), è obbligatorio selezionare il motivo: "Manutenzione" o "Segnalazione", con logging automatico per tracciabilità.

Pagine Donazioni e Segnalazioni

La pagina Donazioni elenca tutte le richieste con stato corrente (pendente, approvata, respinta); cliccando su una voce, si accede al dettaglio per aggiornare lo stato, visualizzare foto e contattare l'utente. Analogamente, la pagina Segnalazioni mostra anomalie/guasti categorizzati, con workflow di moderazione e assegnazione interventi. Entrambe supportano filtri avanzati per data, stato, postazione o categoria.

Analytics e Filtri Globali

La sezione Analytics presenta grafici interattivi (es. barre per utilizzo per locker/parco, linee per trend temporali, torte per categorie), basati su dati aggregati dal database PostgreSQL. Tutte le schede includono filtri di ricerca universali per nome, stato, data o posizione, con sincronizzazione real-time via WebSocket per aggiornamenti immediati.

5. Deployment

Frontend Deployment

Il frontend dell'applicazione UrbanLock è sviluppato in Flutter come applicazione mobile multiplatforma.

Trattandosi di un'app mobile, non è deployata come servizio web, ma viene distribuita tramite build di produzione in formato Android APK.

La build release è stata generata tramite il comando:

```
flutter build apk --debug
```

Il file prodotto è utilizzato per testing interno e validazione funzionale del sistema su dispositivi Android reali.

Disponibile al seguente link:

<https://github.com/UrbanLock/root/tree/main/frontend/apk>

Credenziali di accesso alla console operatore: username `test`, password `1234`

Backend Deployment

Il deployment del backend UrbanLockServer è stato effettuato utilizzando la piattaforma cloud Render, che consente la pubblicazione e gestione di servizi web in modo automatizzato tramite integrazione con repository Git.

Il servizio è configurato come Web Service Node.js ed è attualmente ospitato su un'istanza di tipo Free (0.1 CPU, 512 MB di RAM), sufficiente per scopi didattici e per la fase di testing del progetto.

Per la gestione dei rilasci è stato configurato un Deploy Hook, ovvero un endpoint privato generato dalla piattaforma che consente di attivare manualmente un nuovo deploy del servizio tramite richiesta HTTP autenticata.

Configurazione Build & Deploy

Il sistema è collegato direttamente al repository Git del progetto (branch `main`).

Render è configurato per:

- Eseguire automaticamente il deploy a ogni push sul branch principale (Auto-Deploy attivo).

- Eseguire un Build Command per installare le dipendenze del backend.
- Eseguire uno Start Command per avviare il server Node.js in ambiente di produzione.
- Utilizzare la cartella **backend/** come root directory, trattandosi di un progetto organizzato come monorepo.

Questa configurazione permette un flusso di integrazione continua semplificato: ogni aggiornamento del codice viene automaticamente buildato e messo online senza interventi manuali.

Modalità Sleep

Essendo ospitato su un'istanza gratuita, il servizio è soggetto alla cosiddetta sleep mode prevista da Render.

Dopo un determinato periodo di inattività (assenza di richieste HTTP per alcuni minuti), il server viene automaticamente sospeso per ottimizzare l'utilizzo delle risorse della piattaforma.

Al primo accesso successivo alla sospensione, il servizio viene riavviato automaticamente. Questo comporta un leggero ritardo iniziale nella risposta (cold start), dopodiché il funzionamento torna regolare.

Monitoraggio e gestione

La piattaforma fornisce strumenti integrati per:

- Monitoraggio dei log, utili per il debugging di errori runtime.
- Metriche di utilizzo (CPU, memoria).
- Health checks, per verificare periodicamente lo stato del servizio.
- Gestione delle variabili d'ambiente (es. chiavi JWT, URI MongoDB).

Vantaggi della soluzione adottata

L'utilizzo di Render ha permesso di:

- Automatizzare completamente il processo di deploy.
- Garantire un ambiente accessibile online durante lo sviluppo del frontend.
- Centralizzare monitoraggio e configurazione
- Simulare un ambiente di produzione reale, separato dall'ambiente locale di sviluppo.

In conclusione, la scelta di una piattaforma cloud con integrazione CI/CD ha reso il processo di deployment strutturato, scalabile e conforme alle pratiche moderne di sviluppo software.

Deployment del database

Il database del progetto è stato deployato utilizzando MongoDB Atlas, il servizio cloud ufficiale per l'hosting di database MongoDB.

È stata utilizzata la versione gratuita, adeguata per scopi didattici e per la fase di sviluppo e testing dell'applicazione. Il cluster è stato configurato in modalità condivisa, con risorse limitate ma sufficienti per gestire le operazioni CRUD previste dal sistema (gestione locker, depositi, prestiti, donazioni e segnalazioni).

La scelta di MongoDB Atlas ha permesso di:

- Ottenere un database accessibile via cloud, separato dall'ambiente locale.
- Gestire in modo sicuro le connessioni tramite stringa di connessione protetta (URI con autenticazione).
- Configurare whitelist degli indirizzi IP autorizzati.
- Monitorare utilizzo, performance e stato del cluster tramite dashboard dedicata.

Il backend si connette al database tramite URI memorizzata nelle **variabili d'ambiente**, evitando di esporre credenziali nel codice sorgente.

6. CI/CD (Continuous Integration / Continuous Deployment)

La pipeline CI/CD è implementata tramite GitHub Actions, che consente l'automazione dei processi di integrazione e rilascio del software direttamente a partire dal repository.

Il flusso è configurato per attivarsi automaticamente a ogni push sul branch `main` e prevede le seguenti fasi:

- **Esecuzione automatica dei test backend:**
Vengono eseguiti i test previsti per verificare il corretto funzionamento delle API, dei middleware di autenticazione e della logica applicativa. Questo garantisce che eventuali errori vengano intercettati prima del rilascio in produzione.
- **Build automatica del progetto:**
In caso di superamento dei test, viene avviato il processo di build, che installa le dipendenze e prepara l'applicazione per l'ambiente di produzione.
- **Deploy automatico su Render:**
Una volta completata con successo la fase di build, il backend viene automaticamente deployato su Render, garantendo l'aggiornamento continuo del servizio online.

Questa configurazione permette di adottare un approccio di integrazione continua, riducendo il rischio di errori in produzione e assicurando che il codice presente online sia sempre coerente con l'ultima versione stabile del repository.